Lecture 11: LC-3 Trap Basics and Subroutines

Date: Thursday, June 18th, 2026
Reading: Patt & Patel 3rd ed.: Sec. 5.4.4; Sec. 7.3; Sec. 8.1-8.2

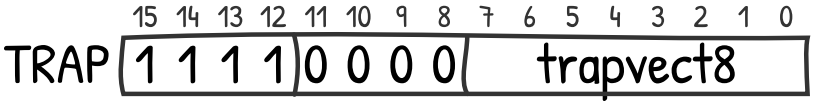
Announcements:

- TBD

Part 1: A Crash Course on LC-3 Traps

TRAPs

To perform operations that require privilege, such as I/O (input/output), LC-3 provides the TRAP instruction, encoded as follows:



A TRAP is a ***system call***: a request for the OS (operating system) to do something privileged. The `trapvect8` is the ***trap vector***, aka the system call number, which identifies the operation the OS should do.

Here are some example system calls and their corresponding trap vectors:

- `0x25` → `HALT`: Stop the clock.
- `0x20` → `GETC`: Wait for a keypress and put its ASCII code into `R0`.
- `0x21` → `PUTC`: Interpret `R0` as an ASCII character and write it to the console.

Let's write a program that prints every character you type, i.e.,

```
while True:
    c = GETC()
    PUTC(c)
```

The code looks like this:

```
.orig x3000
LOOP
getc ; r0 <- ascii code of keypress
putc ; print ascii code in r0 to screen
br LOOP
halt
.end
```

There is one more system call worth discussing:

- `0x22` → `PUTS`: Interpret `R0` as a memory address, specifically the starting address of a null-terminated string of ASCII characters. Print this string to the console.

This is commonly used with the `.stringz` assembler directive. For example, the assembly

```
.orig x4000
.stringz "tom"
.end
```

would initialize memory as follows:

Address	Value
`0x4000`	`0x0074` (ASCII code for `t`)
`0x4001`	`0x006F` (ASCII code for `o`)
`0x4002`	`0x006D` (ASCII code for `m`)
`0x4003`	`0x0000` (null terminator)

Part 2: How a Two-Pass Assembler Works

Under construction

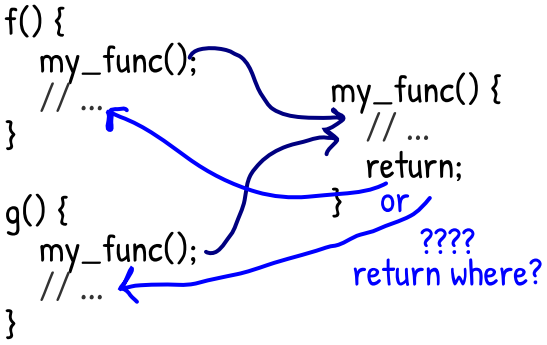
Part 3: Subroutines

Learning objectives *for Lecture 10 Part 2:*

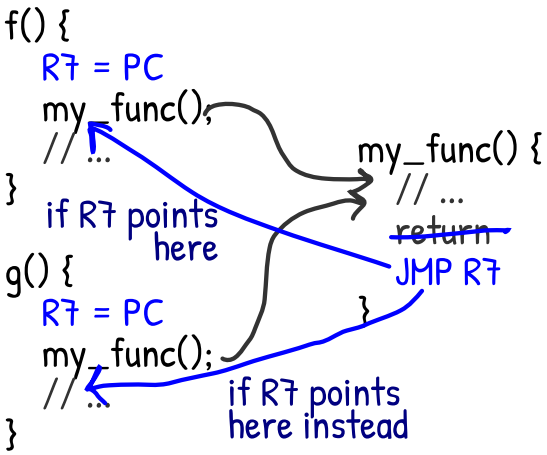
- *Course learning objective #3:* ***Analyze*** *the structure and* ***examine*** *the operation of a basic von Neumann CPU.*
  - ***Write*** *the state transition diagram for executing branch instructions (JMP/BR).*
- *Course learning objective #4:* ***Identify*** *the structure and demonstrate the function of machine language instructions*
  - ***Interpret*** *16 bits as an LC-3 branch instruction (JMP/BR).*
  - ***Simulate*** *LC-3 branch instructions (JMP/BR).*
- *Course learning objective #5:* ***Assemble*** *a symbolic assembly language and* ***describe*** *its structure and purpose.*
  - ***Assemble*** *LC-3 assembly syntax for branch instructions (JMP/BR) into bits.*

The JSRR Instruction

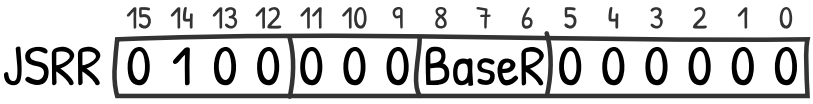
Imagine a program like the following, where two functions `f()` and `g()` both call the function `my_func()`. Changing the PC to point to the code inside `my_func()` is easy enough with the `BR` or `JMP` instructions. Actually, the challenge is implementing the `return` statement inside `my_func()`: **where should it return *to*?** There are at least two options: it can return to `f()`, and it can return to `g()`. A `BR` to either option cannot be safely hard-coded.



To resolve this, we could set a register, say `R7`, to the current `PC` (i.e., the address of the next instruction after the function call) and then implement `return` as `JMP R7`:



At this point, we have derived the behavior of the `JSRR` (**j**ump to the **s**ub**r**outine at the address in the **r**egister operand) instruction, which is encoded as follows:



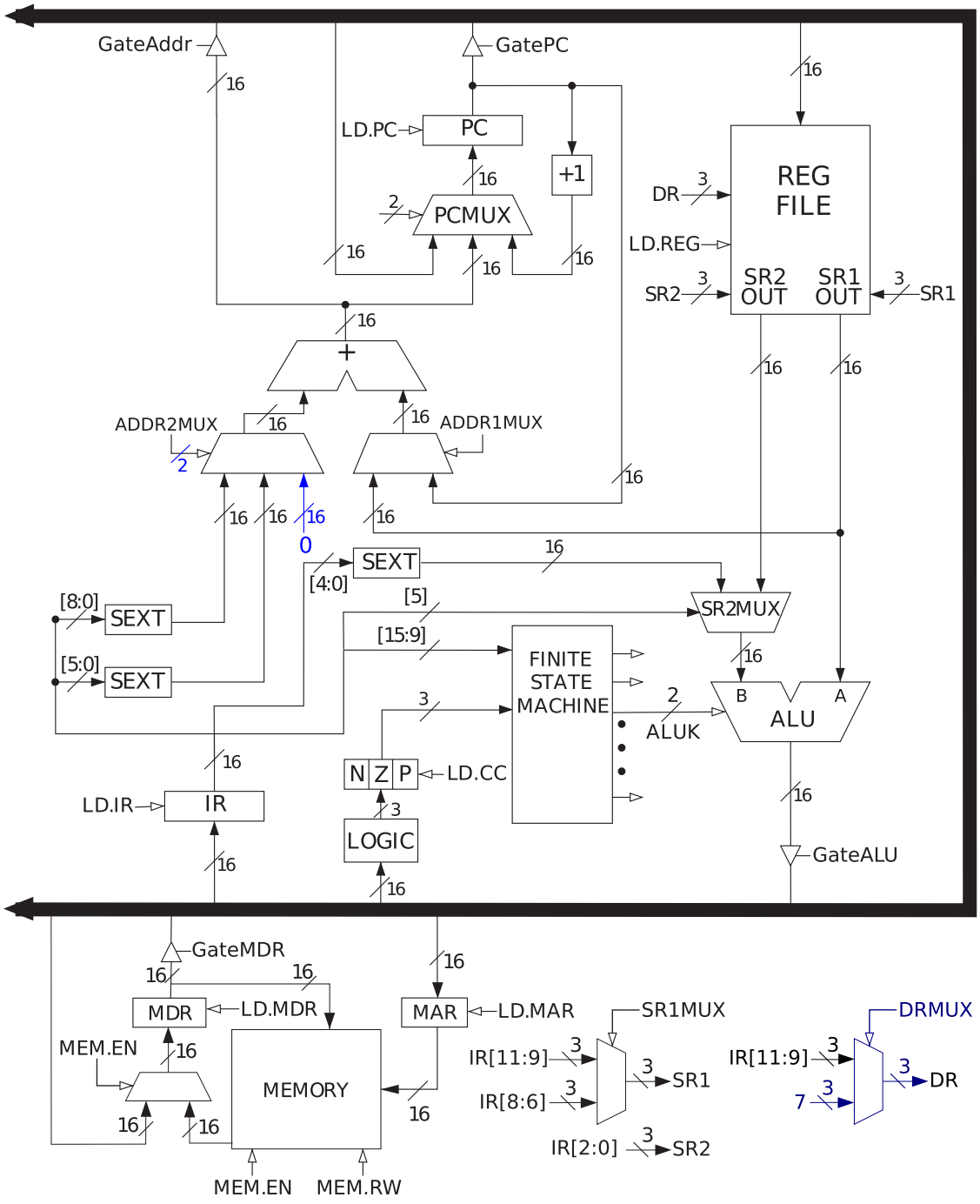
The semantics of `JSRR` are:

In parallel:

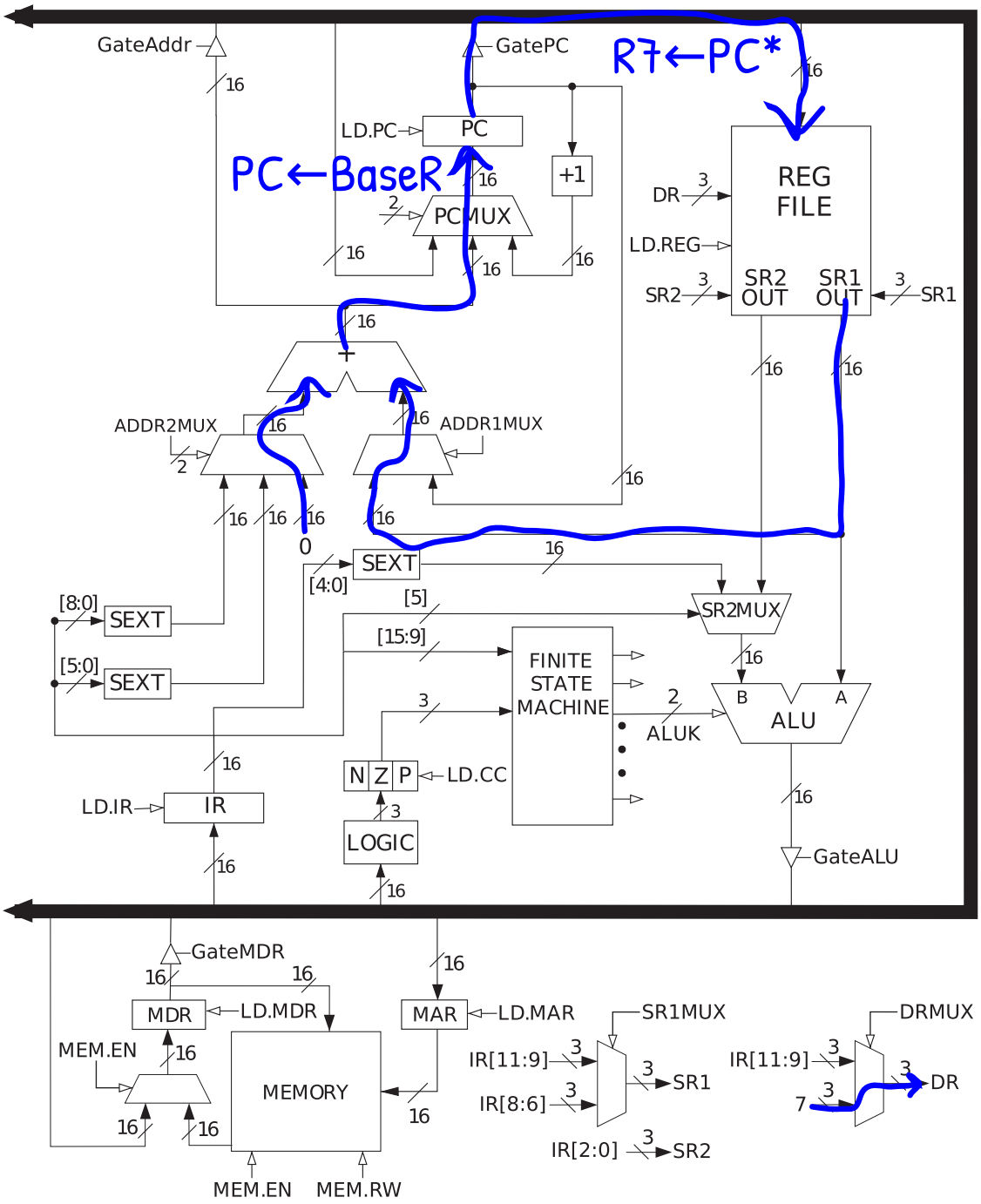
$$R7 \leftarrow PC^{*}PC \leftarrow BaseR$$

As demonstrated above, the `JSRR` saves the ***return address*** in `R7` so that the callee can `JMP R7` to return. In fact, in LC-3 assembly, one can write `RET` as shorthand for `JMP R7`.

To implement `JSRR`, we will introduce a constant `0` input to `ADDR2MUX` and a new `DRMUX` as shown below:



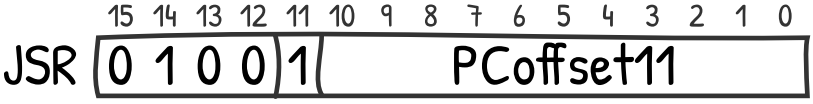
This way, we can update the PC and R7 simultaneously as shown below:



Now, I will show one final branch instruction: an alternate version of JSRR that takes a PC offset instead.

The JSR Instruction

The JSR instruction is encoded as follows:

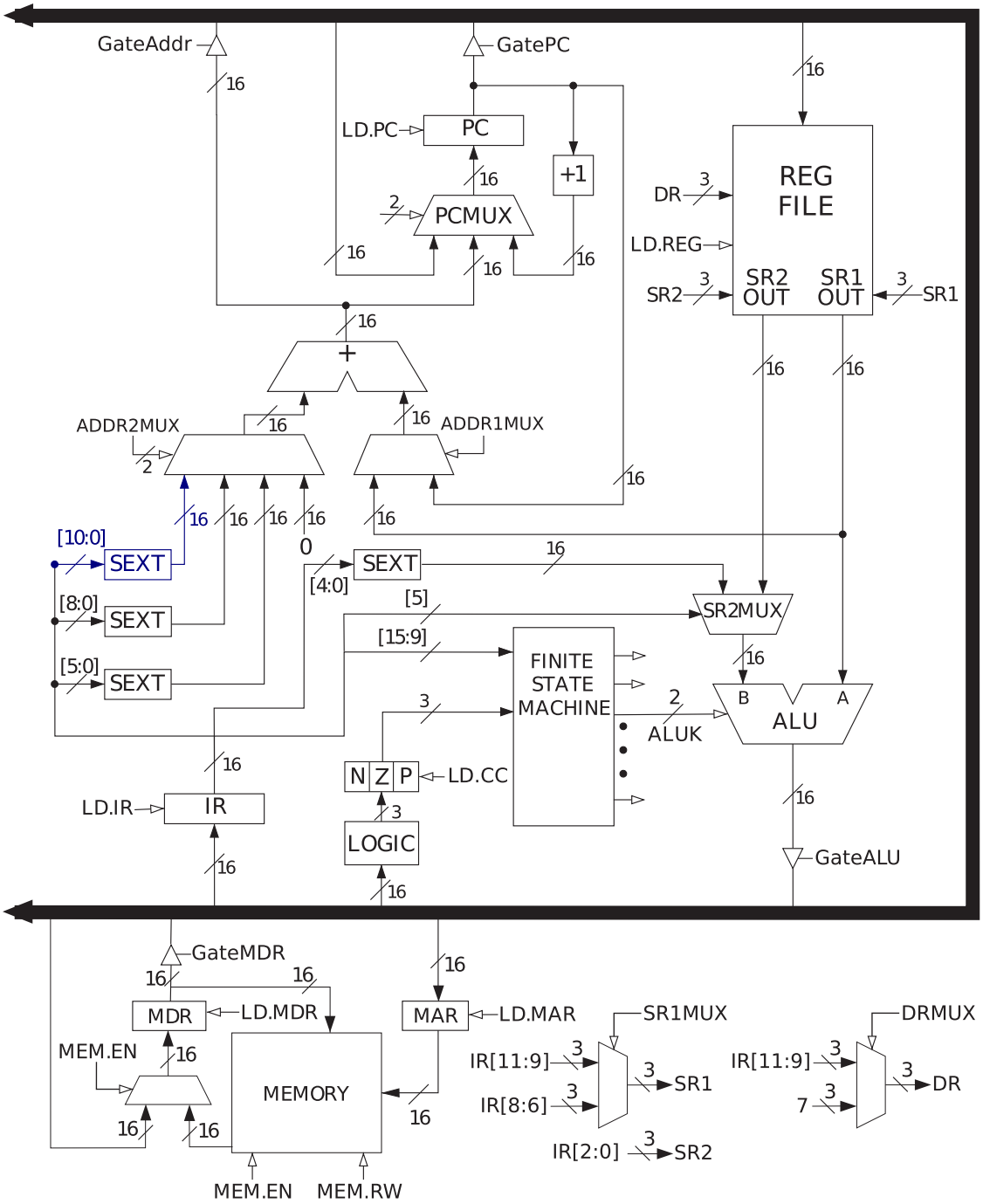


Its semantics follow:

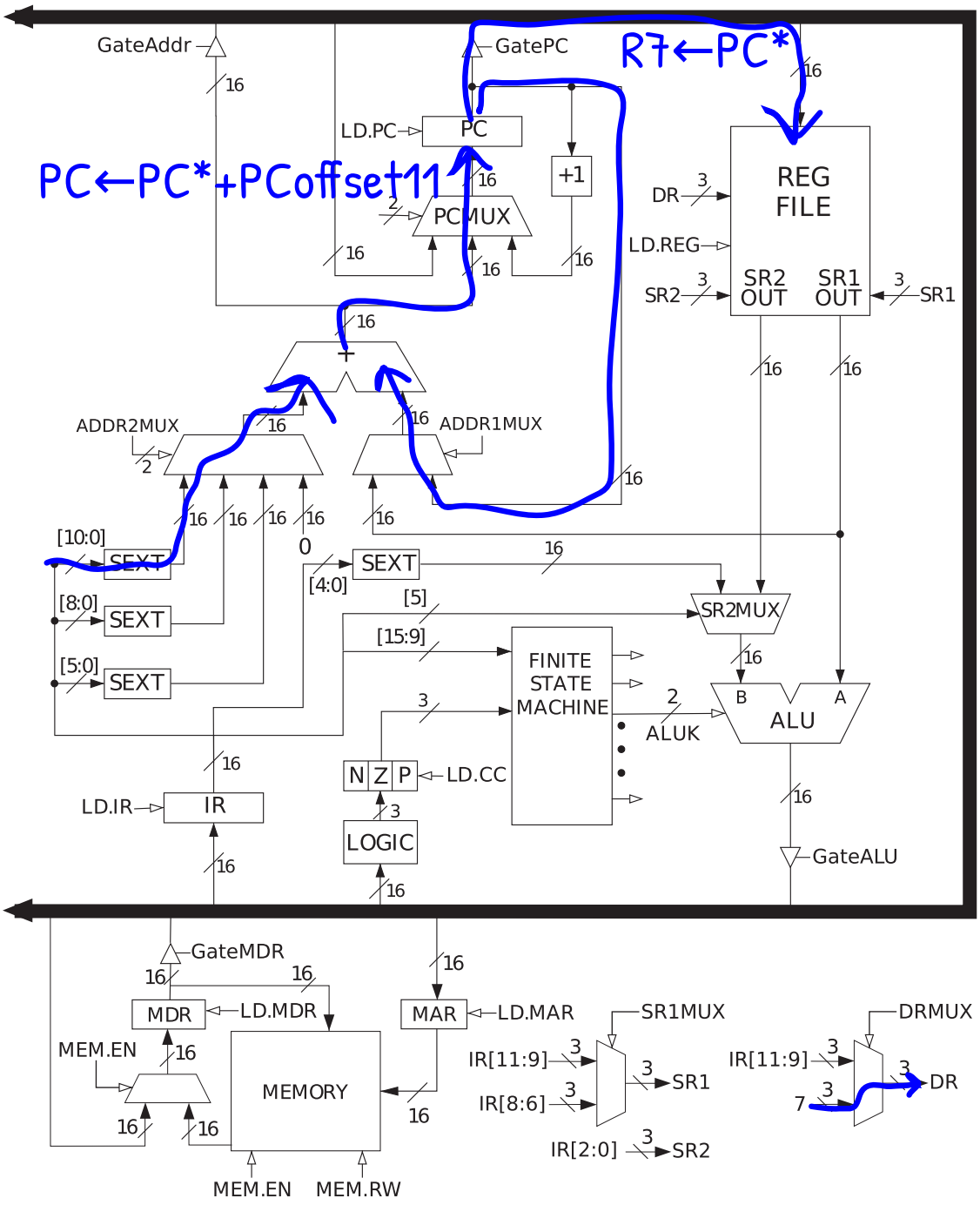
In parallel:

$$R7 \leftarrow PC^* \qquad PC \leftarrow PC^* + SEXT(PCoffset11)$$

Because the `JSR` instruction boasts the biggest PC offset in the whole LC-3 ISA, we need to add dedicated hardware for sign-extending that 11-bit offset:



This addition allows us to put the PC in R7 while also queueing up a new value for the PC (specifically, adding the offset to it):



Just as with `JSRR`, the callee of `JSR` is expected to use `RET` (aka `JMP R7`) to return back to the caller.

Part 4: Linking

Under construction